

AI-Assisted ICU Quality Review Workbench

Final Seminar Report

Software Engineering in the Age of AI (214.3500 / 214.4500)

University of Haifa – Department of Information Systems

Instructor: Prof. Iris Reinhartz-Berger

Team: Mohamed Mhamed, Basil Sheikh, Jumana Qraqra, Hadeel Arafat

1. Research Question, Goals, and Importance

1.1 Research Question

How does structuring AI-based ICU risk prediction as a modular, role-separated software workflow — rather than a single opaque model — affect the reliability, transparency, and auditability of the resulting system, compared with a single-pipeline baseline?

1.2 The Engineering Problem

Clinical decision-support prototypes that use AI are often built as a single, opaque prediction model: a set of inputs goes in, a prediction comes out, with no visibility into how the result was produced, no assessment of input data quality, and no mechanism for a human reviewer to inspect, verify, or challenge the output. In a safety-critical setting such as an Intensive Care Unit (ICU), this is an engineering problem, not only a modelling one. A quality-and-patient-safety coordinator who must prioritize dozens of cases, document review decisions, and maintain an audit trail needs a system that is not just statistically accurate, but transparent, inspectable, and reliable enough to support evidence-based decisions.

This project addresses whether AI-based risk prediction should be delivered as a single opaque model, or restructured as a modular software workflow with role-separated components (data validation, rule-based scoring, weighted clinical assessment, verification with escalation, and explanation). The engineering challenge is to compare these two approaches directly — on reliability, transparency, auditability, and predictive accuracy — rather than assuming that a more elaborate architecture is automatically superior.

1.3 Why This Matters

There is a well-known gap between “a model that predicts well” (measured by accuracy, F1, ROC-AUC) and “a system that can be trusted in a real operational setting.” Bridging that gap is precisely the domain of software engineering: architectural design, validation, testing, and documentation — not algorithmic improvement alone. If a modular workflow can be shown to improve reliability and transparency even when it does not meaningfully improve raw predictive accuracy, that is a meaningful and honest engineering contribution in its own right, and arguably more useful for real deployment than a marginal accuracy gain in a black-box model.

1.4 Goals

- Design and implement a modular, role-separated software workflow for ICU quality review, combining a literature-grounded rule-based baseline (SIRS criteria), a supervised ML risk-prediction model, and supporting AI components (explainability, similar-case retrieval, clustering, anomaly detection, uncertainty estimation).
- Systematically compare the modular workflow against a single-pipeline baseline in terms of predictive accuracy, reliability, and transparency, using the same underlying dataset and the same held-out evaluation.
- Honestly identify and document where the modular design does, and does not, add value — including cases where it does not improve raw prediction performance — rather than assuming that additional AI components are inherently beneficial.
- Deliver a working, documented, and reproducible software artifact (Flask application, SQLite-backed review workflow, trained model, and evaluation harness) suitable for inspection by a course evaluator.

1.5 Expected Contribution

The project contributes a course-level software engineering artifact demonstrating that structuring AI-based risk prediction as a modular, role-separated workflow — rather than a single opaque model — improves the reliability, transparency, and auditability of the resulting system, without necessarily improving predictive accuracy over a single-pipeline baseline. Its contribution is not clinical validation. Its value is an honest, evidence-based comparison showing where modular design helps (traceability, documentation, verification) and where it does not (raw predictive performance), rather than assuming added complexity is inherently better.

2. Engineering Background and Proposed Method

2.1 Overview of the Approach

The system is a Flask-based web application (“AI-Assisted ICU Quality Review Workbench”) intended for an ICU Quality & Patient Safety Coordinator. It combines a review queue, manual case entry, a processed-case library, AI-supported risk analysis, coordinator notes and status tracking, archiving, and an audit trail, backed by a local SQLite store. The system explicitly does not diagnose patients or recommend treatment; its scope is quality-review support and documentation.

2.2 Architecture: Two Parallel Risk-Assessment Pipelines

Two independent pipelines process every case, so that their outputs can be directly compared:

2.2.1 Baseline Pipeline (SIRS-Based)

The baseline implements the Systemic Inflammatory Response Syndrome (SIRS) criteria, a well-established clinical scoring approach (Bone et al., 1992). Four criteria are evaluated: temperature above 38°C or below 36°C; heart rate above 90/min; respiratory rate above 20/min; and white blood cell count above 12,000/mm³ or below 4,000/mm³. The number of criteria met (0–4) is mapped to Low (0–1), Medium (2, SIRS-positive), and High (3–4) risk. Unlike the project's earlier, ad-hoc 10-rule baseline, every threshold in the SIRS baseline is drawn directly from a cited clinical source rather than chosen by engineering judgment alone.

2.2.2 Modular Agent-Inspired Workflow

The second pipeline is a fixed sequential pipeline of specialized components, conceptually inspired by two unrelated bodies of work: AutoGen (Wu et al., 2023), a framework for coordinating multiple specialized LLM-backed agents through structured conversation, and LPW (Lei et al., 2024), a workflow that separates a planning/verification phase from an implementation phase for LLM-based code generation. Importantly, and stated explicitly throughout this project's documentation, the implemented workflow is not a full autonomous, LLM-driven multi-agent negotiation system: there is no independent planning, delegation, or LLM-based reasoning between its stages. What was adopted from AutoGen and LPW is the architectural idea — decomposing a complex task into specialized, inspectable stages with a verification step — not the underlying LLM-agent technology itself. The stages are:

- **Data Validator** – assesses completeness of the ten clinical input fields and labels the case Good, Partial, or Poor quality.
- **Baseline Scorer** – reuses the SIRS-based baseline result as an initial risk level.
- **Weighted Clinical Assessor** – a second, independently weighted rule-based score covering additional physiological and laboratory variables (systolic blood pressure, creatinine, glucose, hemoglobin, SpO₂) beyond the four SIRS criteria, inspired by general severity-scoring concepts such

as APACHE II and SOFA. Its factor weights (1–3 per variable) reflect engineering judgment; they are not calibrated against this dataset or drawn from a specific published scoring table — a stated limitation, not a validated clinical score.

- **Verifier** – compares the weighted score against the baseline level and may escalate the risk level upward (never downward) if the weighted score crosses defined thresholds, or if a co-occurring pair of critical abnormalities is detected.
- **Explainer** – produces a human-readable explanation combining the baseline and weighted factors.

2.3 The Machine Learning Model

A single supervised model, selected between Logistic Regression and Random Forest by F1-score and ROC-AUC on held-out data, is trained on 13 input features (demographic, admission, vital-sign, and laboratory variables). Four columns are explicitly excluded from the model's inputs to prevent label leakage: the target label itself (`hospital_expire_flag`) and three identifier columns (`subject_id`, `hadm_id`, `icustay_id`). Missing numeric values are median-imputed and missing categorical values are imputed with the most frequent category, inside a single scikit-learn preprocessing pipeline; the predicted probability is mapped to Low (<0.33), Medium ($0.33\text{--}0.67$), and High (≥ 0.67) risk bands.

2.4 Supporting AI Components

Beyond the trained model, several supporting components add context for the reviewer without altering the underlying prediction: Explainable AI (saved model feature importances), K-Nearest-Neighbours retrieval of similar cases, K-Means clustering for cohort context, Isolation Forest anomaly detection, and an uncertainty estimate combining predicted probability, entropy, distance from the decision thresholds, and input data quality. An optional generative-AI chat/summary feature calls an external LLM only when an API key is configured; otherwise it falls back to a fully offline, templated summary built from the same structured outputs — the risk-scoring pipeline itself never depends on a live LLM call.

2.5 Tools and Technologies

Python, Flask, scikit-learn, pandas, SQLite (via a lightweight local review-store module), and standard HTML/CSS templates for the interface. Model artifacts (trained pipeline, metrics, feature importances, metadata) are persisted as files and loaded by the running application.

3. Implementation and Evaluation

3.1 Dataset and Preprocessing

The system uses a processed feature table, `patient_features_ai.csv` (1000 ICU stays), derived from the full MIMIC-III v1.4 dataset provided by the course via a shared Google Drive folder — not a separate or unofficial data source. The source tables used were `PATIENTS.csv`, `ADMISSIONS.csv`, `ICUSTAYS.csv`, `CHARTEVENTS.csv` (approximately 33GB), `LABEVENTS.csv`, `D_ITEMS.csv`, and `D_LABITEMS.csv`. Because of the size of the raw tables — `CHARTEVENTS.csv` in particular — the raw files and the original extraction script are not included in the submission package; the processed, ML-ready feature table is included instead. This is stated explicitly as a reproducibility limitation rather than left ambiguous.

The 13 input features cover demographic context (age, gender), admission context (admission type), vital signs (heart rate average/maximum, systolic blood pressure, respiratory rate, temperature, SpO_2), and laboratory values (creatinine, glucose, white blood cell count, hemoglobin). The target label, `hospital_expire_flag`, is used only for training and evaluation, never as a model input. The dataset is

imbalanced: of 1000 stays, 894 survived and 106 died in hospital (10.6% mortality rate), which is why accuracy alone is not treated as a sufficient metric anywhere in this project — precision, recall, F1, and the confusion matrix are reported together.

3.2 Model Evaluation

The selected Logistic Regression model was evaluated at both its default 0.5 probability cutoff and a separately tuned cutoff (0.6136) chosen to maximize F1 on training predictions using a precision-recall curve, then applied unchanged to the held-out test set.

Metric	Default cutoff (0.5)	Tuned cutoff (0.6136)
Accuracy	0.732	0.804
Precision	0.259	0.298
Recall	0.846	0.654
F1-score	0.396	0.410
ROC-AUC	0.838	not separately reported

A data-leakage check (empirically flipping the target label on a subset of rows and confirming predictions were unaffected) passed with zero issues, confirming that the excluded identifier and label columns are not influencing the model.

3.3 Baseline vs. Modular Workflow — Comparative Results

Because the system outputs a Low/Medium/High risk level while the dataset label is binary hospital mortality, results are reported under two interpretations: High-risk detection (only “High” counted as positive) and At-risk detection (both “Medium” and “High” counted as positive).

High-Risk Detection

Metric	Baseline (SIRS)	Modular Workflow
Accuracy	0.86	0.86
Precision	0.34	0.35
Recall	0.33	0.38
F1-score	0.33	0.36
TP / FP / TN / FN	35 / 69 / 825 / 71	40 / 75 / 819 / 66

At-Risk Detection

Metric	Baseline (SIRS)	Modular Workflow
Accuracy	0.674	0.673
Precision	0.193	0.199
Recall	0.651	0.689

Metric	Baseline (SIRS)	Modular Workflow
F1-score	0.297	0.309
TP / FP / TN / FN	69 / 289 / 605 / 37	73 / 294 / 600 / 33

3.4 Discussion of Results

High-risk detection shows a modest improvement for the modular workflow (F1 0.33 → 0.36), and at-risk detection shows a small but consistent improvement as well (F1 0.297 → 0.309). A detailed row-level investigation across all 1000 cases explains why: the Verifier stage only ever escalates risk upward, and it changed the final risk level for just 15 of 1000 cases (1.5%) — 5 cases from Low to High, 4 from Low to Medium, and 6 from Medium to High, with zero de-escalations.

These 15 escalations affect the two evaluation views differently, and the difference is mechanistically traceable rather than coincidental. High-risk detection (only “High” counted as positive) is affected by all 11 escalations that land on High (5 Low→High plus 6 Medium→High). At-risk detection (“Medium” and “High” both counted as positive) is affected only by the 9 escalations that move a case out of the baseline’s Low tier for the first time (5 Low→High plus 4 Low→Medium); the 6 Medium→High escalations do not change the at-risk classification, because those cases were already counted as “at-risk” under the baseline’s Medium label. This is exactly the pattern in the confusion matrices: 9 rows differ between the baseline and modular workflow in the at-risk table (5 move true-negative → false-positive, 4 move false-negative → true-positive), compared with the larger shift underlying the high-risk table.

The escalated subset has a 40.0% true mortality rate versus a 10.6% dataset-wide base rate, a roughly 3.8× enrichment, indicating the verifier’s escalation logic is not indiscriminate. The effect is also concentrated entirely within emergency admissions (700 of 1000 cases); non-emergency admissions show byte-identical metrics between the two pipelines in both views. Given that only 15 rows are affected in total, the exact magnitude of this improvement — in either view — should not be treated as a stable, generalizable effect; the direction (escalated cases are enriched for real risk) is better supported than the precise size of the gain.

The honest conclusion is that the modular workflow’s improvement over the SIRS baseline is real but narrow, and its primary contribution is not predictive accuracy. Its main value lies in reliability and transparency features the baseline does not provide: an explicit data-quality assessment (Good/Partial/Poor), a separate verification step with a documented escalation rationale, a workflow trace, and uncertainty warnings — none of which exist in the single-pipeline baseline. This is precisely the answer the research question was designed to surface: additional AI/workflow structure does not automatically translate into large predictive gains, but it can meaningfully improve auditability and transparency, which is a legitimate and useful engineering outcome in its own right.

3.5 Known Limitations

- Metrics are based on the processed academic dataset and do not constitute clinical validation.
- Class imbalance (10.6% positive label) makes the precision/recall tradeoff central to interpretation; accuracy alone is not meaningful here.
- The weighted clinical assessor’s factor weights are engineering judgment, not calibrated against this dataset or drawn from a single published scoring table.
- Rule-based components (baseline and weighted assessor) do not impute missing values; a missing field simply fails to trigger its threshold, which can silently lower a rule-based risk score rather than being flagged as comparably weighted — unlike the ML model, which uses proper imputation.

- Raw MIMIC-III files and the original feature-extraction script are not included in the submission package due to file size; only the processed feature table is included.

4. Development Process, Challenges, and Lessons Learned

4.1 Major Development Steps

- Initial baseline prototype focused on ICU risk scoring and simple comparison outputs.
- Addition of a modular, agent-inspired workflow with specialized components for validation, risk assessment, verification, explanation, and traceability.
- Faculty feedback identified that the project needed clearer AI content and a clearer real-world target user — the initial framing around a generic clinical user was not specific enough for the assignment's requirements.
- The dataset basis was expanded to 1000 processed ICU stays.
- A real supervised ML model was trained and saved, together with its metadata, metrics, tuned decision threshold, and feature importances — replacing any placeholder or simulated AI output.
- Additional AI support was added: explainability, KNN similar-case retrieval, K-Means clustering, Isolation Forest anomaly detection, and uncertainty estimation.
- Information-system features were added: a review queue, manual case entry, coordinator notes, status and priority tracking, archiving, and an audit trail.
- The target user was re-aligned from a generic clinical role to a specific, appropriate one — an ICU Quality & Patient Safety Coordinator — and the interface was simplified around review work rather than direct clinical decision-making.

4.2 A Significant Design Decision: Correcting Scope Overclaiming

An internal review of the system's own documentation surfaced a mismatch between how the workflow was described and what it actually did: the original wording (“planning-driven multi-agent LLM workflow, inspired by LPW and AutoGen”) implied autonomous, LLM-driven agent reasoning, while the actual implementation is a deterministic, rule-based sequential pipeline with no LLM call anywhere in the risk-scoring path. This was treated as a correctness issue, not a cosmetic one, and addressed directly: every instance of this wording across the codebase, in-app text, README, and user guide was located and replaced with an accurate description (“modular, agent-inspired workflow ... not a full autonomous AutoGen-style multi-agent negotiation system”), and an explicit clarifying sentence was added wherever the workflow is described to a reader.

The same review also corrected evaluation language that had previously implied the modular workflow improves predictive performance in general terms; the revised wording states plainly that the model shows high recall but low precision, that the workflow's main contribution is transparency, traceability, documentation, and auditability rather than accuracy, and that the system does not provide diagnosis or treatment advice.

4.3 Establishing a Literature-Grounded Baseline

A related design issue was identified when justifying the baseline pipeline's specific thresholds: the original 10-rule baseline used clinically plausible but uncited threshold values, chosen by engineering judgment rather than derived from a specific published source. This was corrected by replacing the baseline with the four SIRS criteria (Bone et al., 1992), a well-established and directly citable clinical scoring approach whose required input variables (temperature, heart rate, respiratory rate, white blood cell count) were all already

present in the dataset. The weighted clinical assessor, whose broader set of variables and specific weights do not correspond to a single published scoring system, is now explicitly documented as an engineering extension inspired by general severity-scoring concepts (APACHE II, SOFA), not a validated clinical score — an intentional, disclosed distinction between what is literature-grounded and what is the team's own design choice.

Because the Verifier's escalation logic starts from the baseline's Low/Medium/High classification rather than from an independent scoring rule, replacing the baseline with SIRS also shifted which cases the Verifier escalates. The team re-ran the full evaluation harness after this change and re-verified both the high-risk and at-risk comparison tables against the current codebase (Section 3.3), rather than assuming the earlier ad-hoc-baseline results still applied.

4.4 Data Provenance and Reproducibility

A related challenge concerned documenting exactly where the processed dataset came from. The team confirmed that `patient_features_ai.csv` was derived from the full MIMIC-III v1.4 dataset supplied by the course through a shared Google Drive folder — including `CHARTEVENTS.csv` and `LABEVENTS.csv`, which were not present in the smaller ZIP subset initially examined — rather than from an unofficial or unclear source. Because `CHARTEVENTS.csv` alone is approximately 33GB, the raw tables and the original extraction script could not be included in the submission package; this is now stated explicitly as a disclosed reproducibility limitation, rather than left implicit or overstated.

4.5 Fixing a Real Software Defect

A concrete engineering defect was found and fixed in the manual case-entry form: submitting non-numeric text in a numeric field (for example, entering text in the age field) previously raised an uncaught exception and produced an unhandled server error. The fix separates parsing from validation: empty fields remain permitted (handled by the existing imputation pipeline), while non-empty invalid values are caught, reported to the user with a field-specific message, and the form is re-rendered with the user's input preserved — with no case created and no prediction run until the input is valid.

4.6 Lessons Learned

- Architectural inspiration and implementation reality must be described with equal precision: describing a rule-based pipeline as an autonomous LLM-agent system, even if unintentional, is a correctness issue that a careful reader (or evaluator) will notice.
- A rule-based clinical scoring component should be grounded in a specific, citable source wherever the required variables are available in the data; where that is not fully possible, the gap between literature-grounded and team-designed components should be stated explicitly rather than left ambiguous.
- Comparing two pipelines side-by-side on identical held-out data, rather than reporting only one pipeline's metrics, is what allowed a small but genuine effect (the 15 escalated cases) to be found, quantified, and explained — an unexamined single-pipeline evaluation would not have surfaced this.
- When one pipeline component (here, the baseline) is changed, every downstream component and evaluation table that depends on it must be re-run and re-checked, not assumed to be unaffected — the Verifier's dependence on the baseline's classification is a clear example of this.
- Reproducibility gaps (a missing extraction script, an unconfirmed exact row count for a processed file) are best handled by disclosing them precisely rather than omitting the issue or overstating what is actually reproducible.

- Basic input validation (numeric parsing, empty-value handling) is easy to overlook in an academic prototype but is exactly the kind of defect a course evaluator is likely to find by simply using the form.

5. Summary and Future Work

5.1 Key Conclusions

This project set out to examine whether structuring AI-based ICU risk prediction as a modular, role-separated software workflow — rather than a single opaque model — improves reliability, transparency, and auditability, compared with a single-pipeline baseline. The evidence supports a nuanced but clear answer: the modular workflow does not meaningfully improve raw predictive accuracy over the SIRS-based baseline in either evaluation view (the effect is real but narrow, driven by 15 of 1000 cases, concentrated in emergency admissions). Its genuine and consistent contribution is in reliability and transparency — explicit data-quality assessment, a documented verification/escalation step, a workflow trace, and uncertainty warnings — none of which the single-pipeline baseline provides.

5.2 Strengths

- A real, trained, and properly evaluated supervised ML model with leakage checks, tuned decision thresholds, and documented feature importances — not a simulated or placeholder AI output.
- A literature-grounded baseline (SIRS criteria, explicitly cited) rather than an uncited set of ad-hoc thresholds.
- A direct, side-by-side comparison between baseline and modular workflow on identical held-out data, with the source of any difference traced to specific rows and mechanisms rather than asserted.
- Honest, explicit documentation of scope and limitations throughout — including correcting an earlier overclaiming of LLM-agent capability, and re-verifying comparison results after the baseline itself was revised.

5.3 Weaknesses and Limitations

- The weighted clinical assessor's variable weights are not calibrated against the dataset or a single published scoring table.
- Rule-based components handle missing values by simply not triggering a threshold, rather than through proper imputation, unlike the ML model.
- The improvement attributable to the modular workflow rests on a small number of affected cases (15 of 1000), limiting how confidently its exact magnitude can be generalized, in both the high-risk and at-risk views.
- The original raw-data extraction script and raw MIMIC-III tables are not included in the submission package, limiting end-to-end reproducibility from raw data.

5.4 Directions for Future Work

- Calibrate the weighted clinical assessor's factor weights empirically against the dataset (for example, via logistic regression coefficients or another data-driven weighting scheme) instead of relying on engineering judgment alone.
- Extend the evaluation to a larger and/or external ICU cohort to test whether the emergency-admission-specific improvement generalizes beyond this dataset.

- Investigate a genuine LLM-based planning/verification layer (in the spirit of AutoGen or LPW) as an explicit extension, clearly distinguished from the current deterministic implementation, and evaluate whether it adds capability beyond the rule-based verifier.
- Publish or preserve the raw-data-to-feature-table extraction script (even if the raw files themselves remain restricted) to close the current reproducibility gap.
- Add automated unit tests for the core rule-based and verification logic, to formalize the manual testing performed during development, and add a regression test that re-checks the baseline-vs-modular comparison tables whenever either pipeline changes.

6. User Manual

6.1 System Requirements and Dependencies

- A local Python 3 environment with virtual-environment support.
- A web browser to access the local Flask application.
- Dependencies installed from requirements.txt (Flask, pandas, scikit-learn, numpy, joblib, and related packages). No external database setup is required — SQLite storage is created automatically at runtime.

6.2 Installation and Running (PowerShell)

```
cd Software_System
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

Then open <http://127.0.0.1:5000/> in a browser.

6.3 Installation and Running (Git Bash)

```
cd Software_System
python -m venv .venv
source .venv/Scripts/activate
pip install -r requirements.txt
python app.py
```

6.4 Required Inputs

The coordinator can select an existing processed ICU case from the case library, or enter a manual case with the available demographic, admission, vital-sign, and laboratory fields. Missing values are permitted and handled by the preprocessing/model pipeline; non-numeric entries in numeric fields are rejected with a clear, field-specific message rather than causing an error.

6.5 Automatic Steps (per case)

- Load processed features and align entered values to the trained model's input schema.

- Apply imputation and preprocessing consistent with the training pipeline.
- Generate the supervised ML risk probability and Low/Medium/High risk band.
- Generate explanation, similar-case, cluster, anomaly, and uncertainty context.
- Run the SIRS baseline and the modular workflow (validation, weighted scoring, verification, explanation) in parallel.
- Create or update the corresponding workflow record and audit-trail entry.

6.6 Manual Steps (Coordinator Workflow)

- Open the Quality Review Queue or the Case Library.
- Open an existing case or create a new review ticket.
- Review the AI-supported risk summary, explanation, and uncertainty indicators.
- Set review status (New, In Review, Needs Follow-up, Ready for Quality Team Review, Reviewed) and priority.
- Add a coordinator note.
- Archive the case once it has been reviewed and documentation is complete.
- Inspect the audit trail for traceability and follow-up.

6.7 Outputs

- Risk probability and Low/Medium/High review-support risk band.
- Confidence and uncertainty indicators, including data-quality flags.
- Top model explanation signals (feature importances).
- Similar-case, cluster, and anomaly context.
- Review notes, status, priority, archive state, and full audit history.

6.8 Repository / Code Structure

```
Software_System/
  app.py – Flask routes and request handling
  review_store.py – SQLite-backed workflow storage (review items, manual
cases, notes, audit events)
  agents/ – AI components: ML prediction, explainability, similar cases,
clustering,
      anomaly detection, uncertainty, modular workflow, optional
generative-AI summary
  baseline/ – SIRS-based baseline pipeline
  ml/ – model training script
  evaluation/ – baseline vs. modular workflow evaluation harness
  models/ – saved model, metrics, metadata, feature importances
  data/ – processed feature table and evaluation results
  templates/, static/ – user-facing HTML/CSS
  requirements.txt, README.md – dependencies and documentation
```

6.9 What the System Does Not Do

The system does not diagnose patients, does not recommend treatment, medication, or emergency action, and does not replace qualified clinical staff or hospital policy. It supports quality-review prioritization, documentation, and auditability only.

7. Related Literature

7.1 Article Summaries

Wu et al. (2023) — AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation

AutoGen is an open-source framework (Microsoft Research) for building LLM applications through multiple customizable, conversable agents that coordinate by exchanging messages to accomplish a task. Its central contribution is “conversation programming”: defining agents with specific roles and capabilities (backed by LLMs, tools, or humans), and programming how they interact, so that complex workflows such as code generation, retrieval-augmented question answering, and multi-user collaboration can be built from reusable, composable roles rather than a single monolithic model call. The paper demonstrates this across six applications and reports consistent gains over single-agent baselines.

Lei, Chang, Lipovetzky, and Ehinger (2024) — Planning-Driven Programming: A Large Language Model Programming Workflow (LPW)

LPW proposes a two-phase workflow for LLM-based code generation: a solution-generation phase that produces a natural-language solution plan and then verifies that plan step-by-step against visible test cases, followed by a code-implementation phase that drafts code from the verified plan and, if the code fails a test, compares its execution trace against the plan's expected intermediate outputs to generate a precise, targeted refinement. The key idea is separating “what should happen, verified against evidence” from “how it is implemented,” so that debugging is guided by a validated intended solution rather than by trial and error. LPW is reported to achieve new state-of-the-art Pass@1 accuracy across several code-generation benchmarks.

MIMIC-III Benchmarks / ICU Prediction Tasks (Harutyunyan et al., 2019 and related benchmark literature)

This body of work defines standardized, reproducible ICU prediction tasks on MIMIC-III — in-hospital mortality prediction, decompensation detection, length-of-stay prediction, and phenotype classification — emphasizing consistent cohort definitions, explicit train/test splits, and transparent, comparable evaluation metrics rather than ad-hoc, one-off evaluations.

MIMIC-Extract / MIMIC Data Preprocessing Frameworks

This literature addresses how raw, irregularly sampled MIMIC event tables (such as CHARTEVENTS and LABEVENTS) are transformed into patient-level or time-windowed feature tables suitable for modelling, with particular attention to missing-value handling, feature consistency across patients, and preventing label leakage from identifier or outcome columns into the model's inputs.

ICU Severity Scoring and Clinical AI Workflow Literature (SIRS, APACHE II, SOFA, and Clinical Decision-Support / Explainability Discussions)

Established ICU severity-scoring systems — SIRS (Bone et al., 1992), APACHE II, and SOFA — combine physiological, laboratory, organ-function, and admission-context variables into interpretable, threshold-based scores. Related discussions of clinical AI workflow design emphasize that deployed clinical AI tools

require workflow integration, transparency, human oversight, clear communication of uncertainty, and carefully bounded claims about what the tool can and cannot do, rather than functioning as an unexplained black box.

7.2 Contribution of Each Article to This Project

AutoGen

The implemented workflow is architecturally inspired by AutoGen's core idea of decomposing a complex task into specialized components with distinct responsibilities. However, the project is explicit about its scope: the software uses a modular, fixed sequential pipeline of components (validator, scorer, weighted assessor, verifier, explainer), not a full autonomous, LLM-driven, AutoGen-style multi-agent negotiation system. AutoGen contributed the architectural vocabulary and design inspiration for this decomposition, not the underlying LLM-conversation technology, which was deliberately not used in the risk-scoring path for a safer, more inspectable academic prototype.

LPW

LPW's separation between a verified plan and its implementation informed the design of the workflow's Verifier stage, which compares the weighted clinical assessment against the baseline result and applies a documented, inspectable escalation rule before finalizing a risk level — conceptually analogous to LPW's plan-verification-before-implementation principle. As with AutoGen, this is an architectural analogy: the comparison is performed with fixed rule-based logic, not an LLM-based verification step as in LPW's original code-generation setting.

MIMIC-III Benchmarks / ICU Prediction Tasks

This literature supports the project's decision to evaluate ICU risk prediction with an explicit label (hospital_expire_flag), a proper train/test split, and multiple complementary metrics (precision, recall, F1, ROC-AUC, confusion matrix) rather than accuracy alone — directly shaping the evaluation methodology used in Section 3.

MIMIC-Extract / MIMIC Preprocessing Frameworks

This work supports the project's approach to using a processed, ML-ready feature table (rather than raw MIMIC files) and documenting exactly which columns were excluded as identifiers or as the label itself, and how missing values are handled — directly reflected in the Dataset & Preprocessing section of the system and this report.

SIRS / APACHE II / SOFA / Clinical AI Workflow Literature

SIRS provided the specific, citable clinical criteria adopted for the baseline pipeline, replacing an earlier set of uncited, ad-hoc thresholds. APACHE II and SOFA provided general conceptual grounding — not a specific formula — for the weighted clinical assessor's broader variable set, a distinction the project states explicitly rather than blurring. The broader clinical AI workflow literature reinforced the project's emphasis on transparency, explicit uncertainty communication, and carefully bounded clinical claims (“quality-review support only — not diagnosis or treatment”) throughout the system's design and documentation.